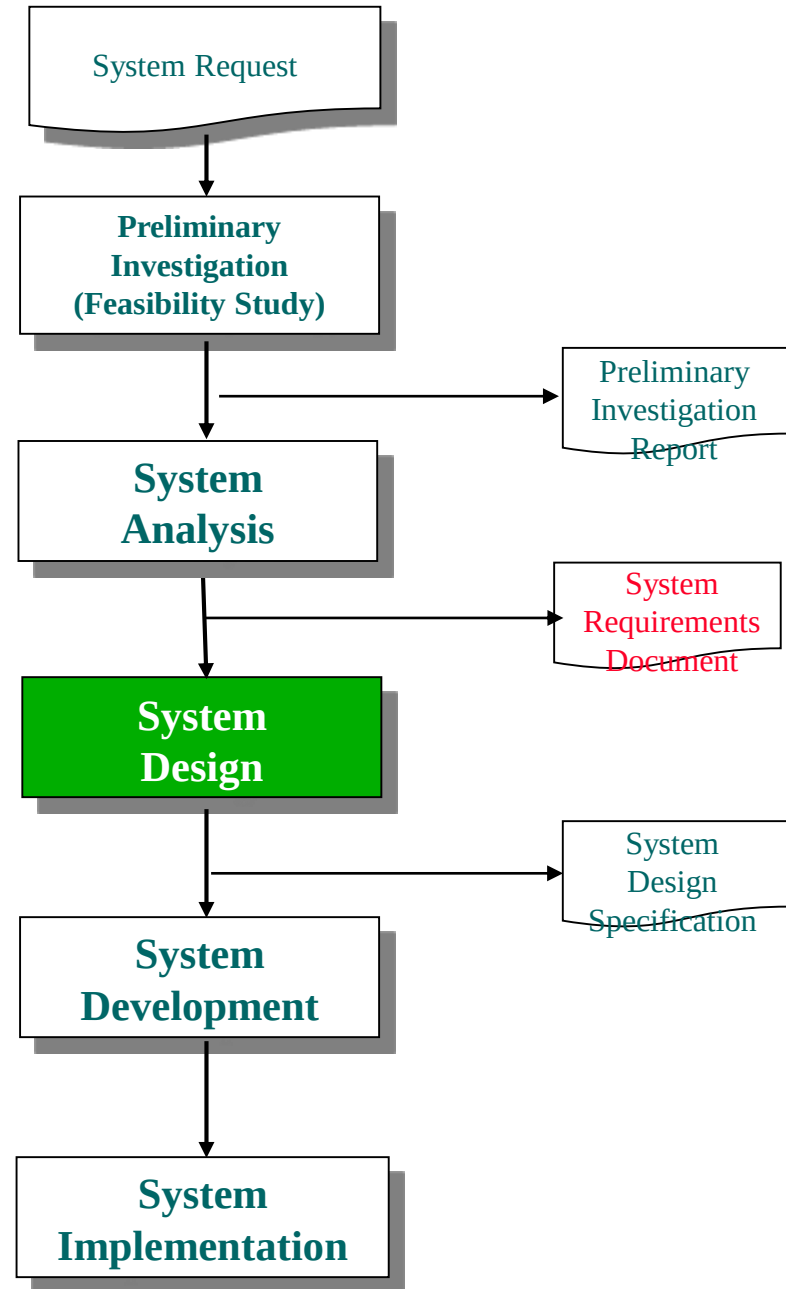


Chapter 8

Procedural Design

SDLC



Program Level Design Techniques

- **System Level.** A System Design tool (eg DFD, ERD, Structure Chart) is used.
- **Program Level.** After 'explosion', the DFD at the **lowest** level is a **process or a program** which cannot be broken down further.
- **Design Techniques.** Variety of design techniques are available to describe a Program (see next slide)

Main Program Design Techniques

1. Program Flowchart
2. Box Diagram
3. Structured English
4. Pseudocode
5. Decision Tables
6. Decision Trees



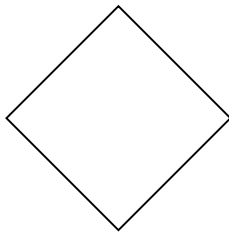
Main Focus

1. Program Flowchart

- **Graphical** tools provide excellent description of procedural detail.
- Widely used in the **past**.
- **Notation** (examples) :



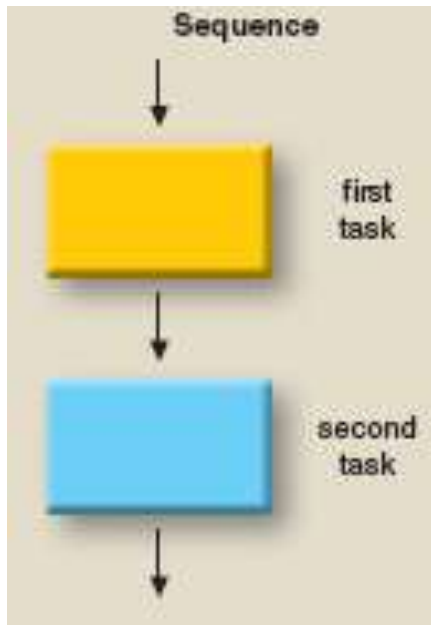
denotes an **action**
(process) step



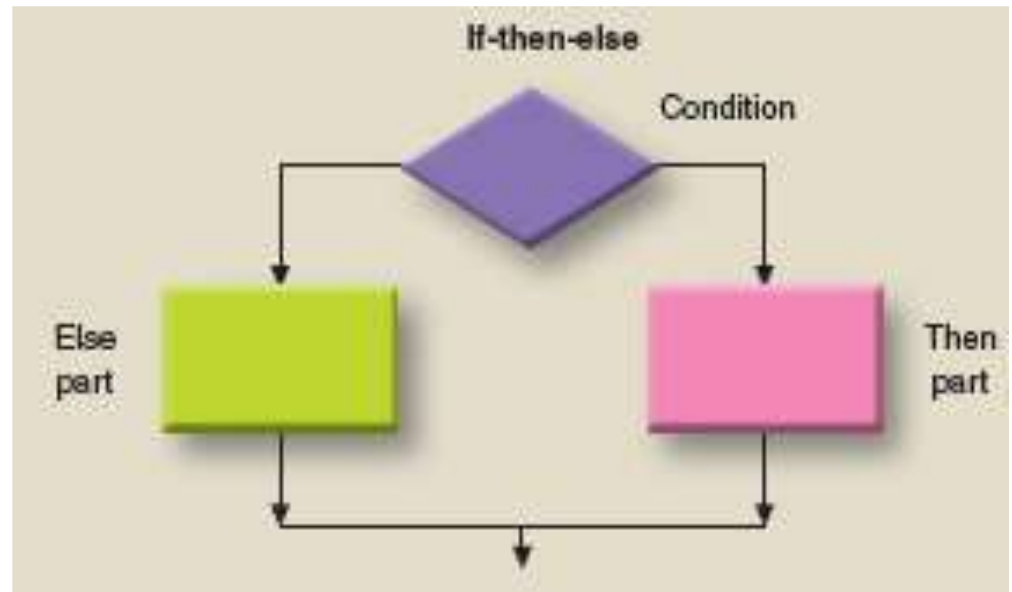
denotes a **condition**
step

Examples of Flowchart

Sequence Structure

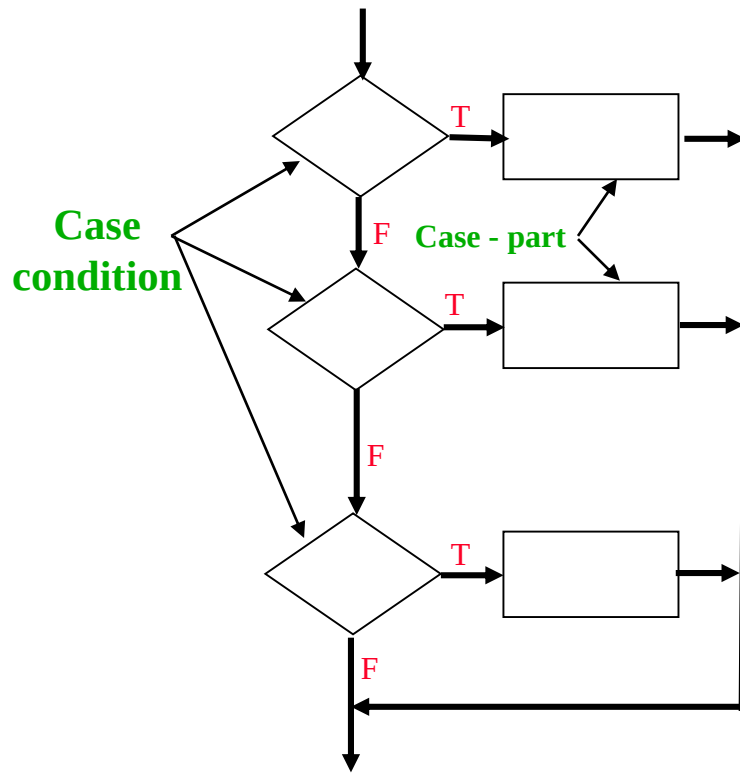


Selection Structure

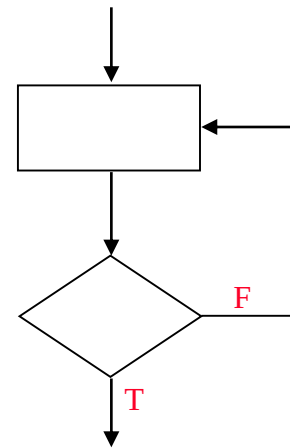


Examples of Flowchart

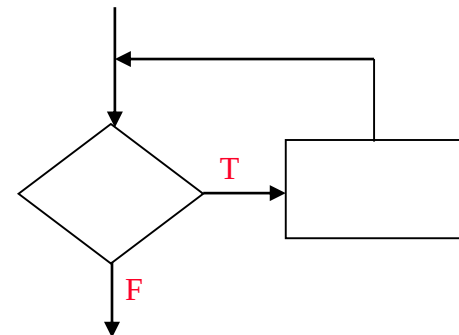
Selection



Repetition / Looping



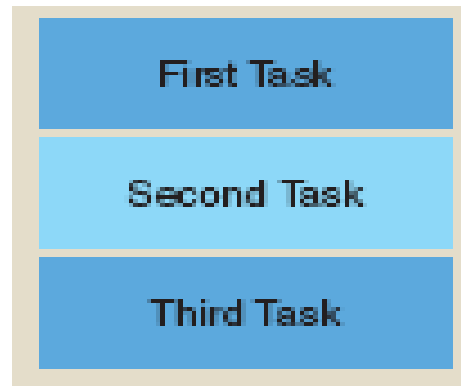
Repeat - until



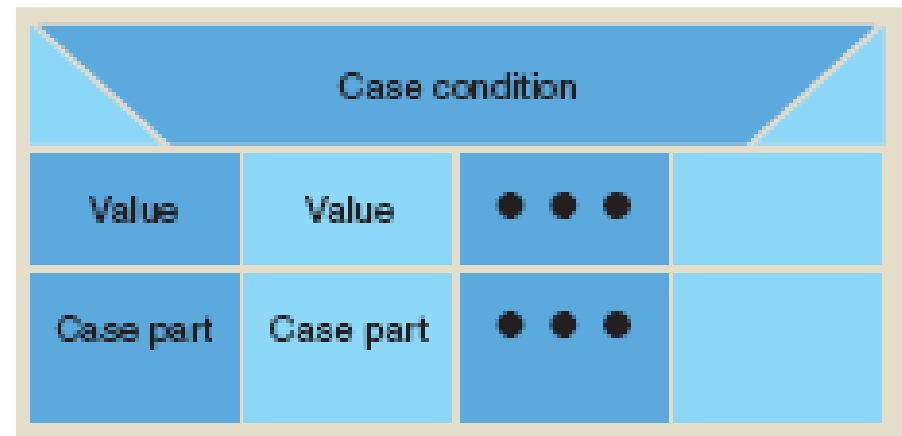
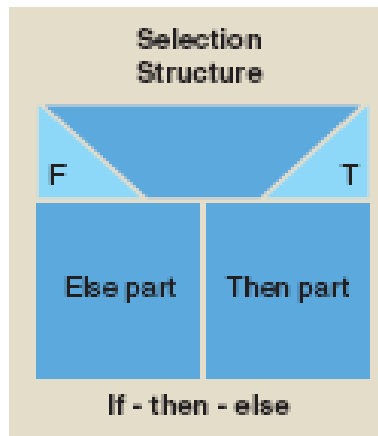
Do - while

2. Box Diagram

- Sequence Structure

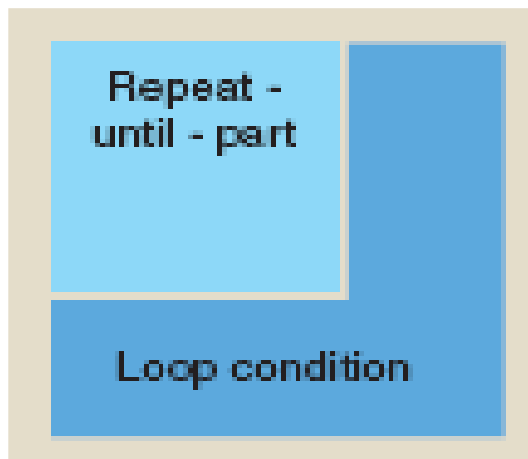
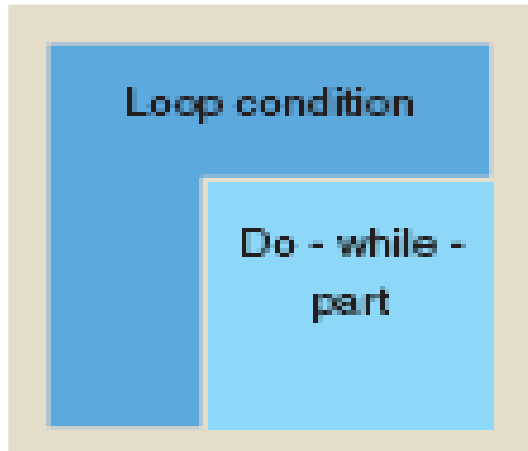


- Selection / Case Condition

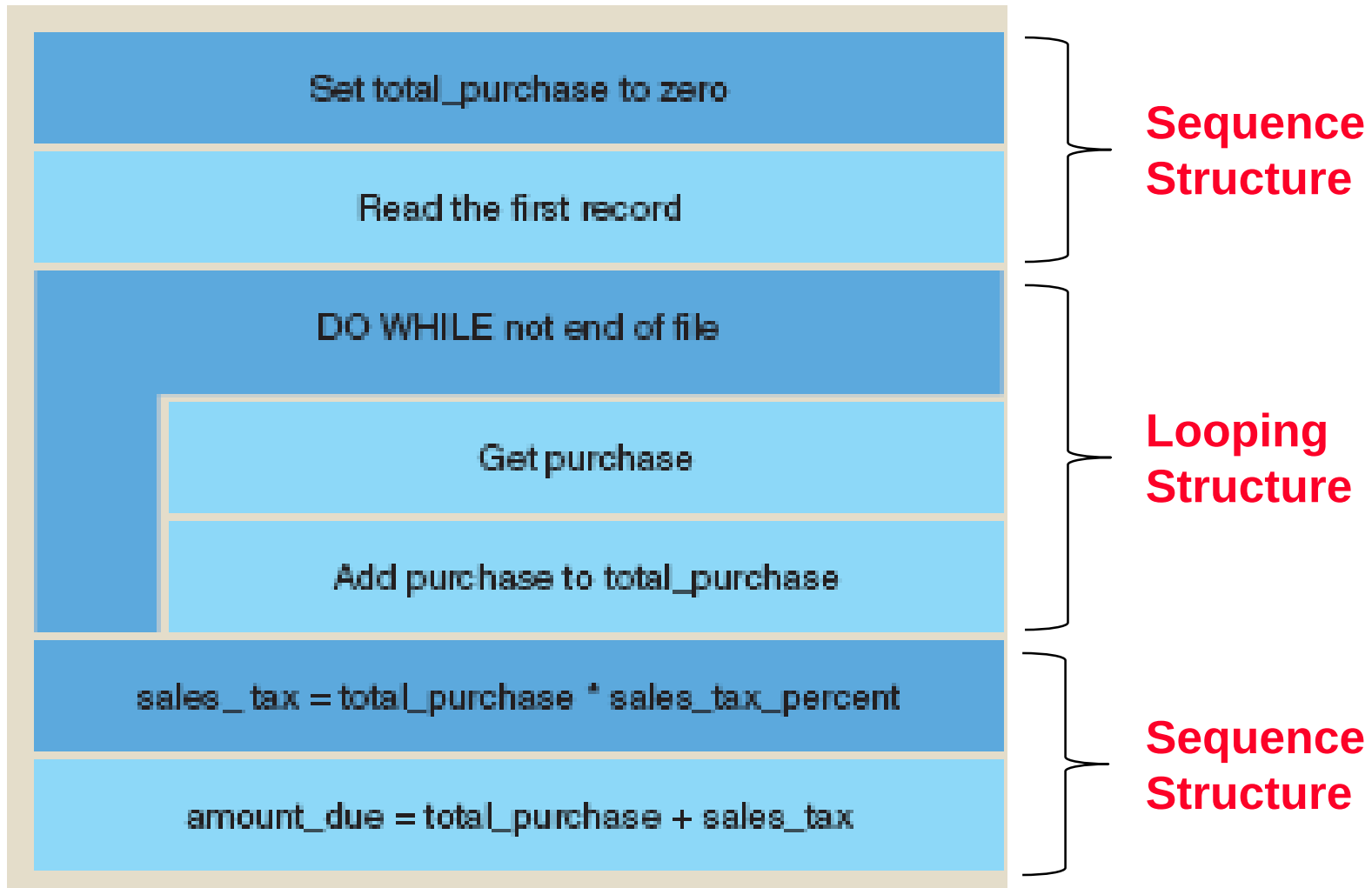


2. Box Diagram

- Looping / Repetition Structure



Box Diagram Example - Processing Purchases



3. Structured English

- **Features** - Like spoken English. Used to describe program operations and processes.
- **Weakness**
 - Limited and restricted *vocabulary* available.
 - Tends to be wordy, *incomplete*, confusing and ambiguous.
- **Strengths**
 - Programmers can focus on the program *logics* and less on *syntax*.
 - Users may find it to *understand* and to *verify* requirements.

Eg 1: Sequence Structure

Customer Order Processing

Obtain customer details

Subtract one from stock on hand

Print customer invoice

Eg 2: Looping Structure

Calculate daily total orders and updating of customer records

Looping
Structure

```
ASSIGN daily-total = 0
```

```
DO WHILE there are orders in ORDERS
```

```
    READ next order in ORDERS
```

```
    DISPLAY invoice-number, customer-name, total-amount
```

```
    CALCULATE daily-total = daily-total + total-amount
```

```
ENDDO
```

```
DISPLAY daily-total
```

Looping
Structure

```
REPEAT UNTIL (end of file)
```

```
    Update CUSTOMER record
```

```
ENDREPEAT
```

Eg 3: Selection Structure

Deciding on % of discount on the quantity ordered

```
ASSIGN daily-total = 0
```

```
IF (quantity ordered is  $\geq 50$ ) THEN
```

```
    Allow 5% discount
```

```
ELSE (quantity ordered is  $\geq 10$  AND  $\leq 49$ )
```

```
    Allow 2% discount
```

```
ENDIF
```

Exercise for Student

- A particular bank has set certain *conditions* for processing of a loan application approval.
 - If a customer **has an account** with the bank and there is **no outstanding loan**, then a loan will be **approved**.
 - If a customer has an account with the bank but some amount is outstanding from previous loans then the loan will be approved if *management approval* is given.
 - **Reject** all loan applications in all other cases.

- (a) Write the above in **Structured English**.
- (b) How many '**conditions**' and '**actions**' in the above?



Conditions



Actions

Solution

Structured English

3 Conditions

2 Actions

```
IF customer has an account THEN
  IF customer has no outstanding loan THEN
    Approve loan
  ELSE
    IF management approval is given THEN
      Approve loan
    ELSE
      Reject loan application
    ENDIF
  ENDIF
ELSE
  Reject loan application
ENDIF
```

The diagram illustrates the mapping between the structured English code and the annotations on the left. Arrows point from the annotations to the corresponding parts of the code:

- 3 Conditions** (green text) points to three green-bordered boxes in the code: "customer has an account", "customer has no outstanding loan", and "management approval is given".
- 2 Actions** (red text) points to two red-bordered boxes in the code: "Approve loan" and "Reject loan application".

Disadvantages of SE

- **Not Easy To Use.** Unrestricted and indiscriminate use of borrowed commands.
- **For Small Activities.** Structured English is only suitable for describing small and specific activities or functions.

4. Pseudocode

- **Borrowed vocabulary.** Pseudocode is similar to and borrowed from a real program code eg. PASCAL or C++.
- **Supplement.** May borrow the style and syntax from a programming language and a natural language (eg English)
- **Not language dependent.** The programmer does not have to be concerned with language-dependent details eg variable declarations, system-specific codes and sub-routines.

Example Pseudocode

```
IF condition-1 THEN
    IF condition-2 THEN
        PERFORM block-a
    ELSE
        PERFORM block-b
    ENDIF
ELSE
    PERFORM block-c
ENDIF

DO calculate_sales-tax
    IF item is exempted from tax THEN
        sales_tax = 0
    ELSE
        IF item_price < RM100 THEN
            sales_tax = item_price * 0.3
        ELSE
            IF item_price < RM200 THEN
                sales_tax = item_price * 0.4
            ELSE
                sales_tax = item_price * 0.5
            ENDIF
        ENDIF
    ENDIF
ENDIF
ENDDO
```

Advantages of Pseudocode

- **More Time on Logics.** As pseudocode does not follow any programming syntax rules, the programmer would then have more time to focus on correctness of logics and algorithms.
- **Supplement.** The programmer may supplement the pseudocode with some natural language remarks or descriptions.
- **Language Independent Details.** The analyst need not have to be concerned with language-dependent details such as variable declarations, system-specific codes and sub-routines. Focus only on algorithm design.

Disadvantages

- **Double Coding.** The programmers would start writing the pseudocode (in a programming language) and then translate it. Hence the program is written twice, once in 'pseudocode' and another in real code.
- **Wrong Focus.** Using pseudocode, the programmer may focus too much on *coding* rather than *design* or logics.
- **Non standard.** Each programmer has his style and programming language background, writing non-standard pseudocode.

Decision Tables

What are Decision Tables

- **Definition**

- Also known as *cause-effect charts*
- Used as a means of describing a complex processes in which the **actions** to be taken depend upon several different **conditions**.
- Also useful in system testing, eg in developing test cases or scripts.

- **General Format**

- 4 quadrants

Condition Heading	Decision Rule Numbers
Condition Statements	Condition Entries
Action Statements	Action Entries

Learn to Read the Decision Table

	1	2	3	4
Is product in stock ?	Y	Y	N	N
Is credit status OK ?	Y	N	Y	N
Accept order	X			
Reject order		X	X	X

How many conditions are there ?

How many actions are there ?

How many combinations are there ?

If product in stock but credit status not ok, what action to be taken ?

Condition Heading	Decision Rule Number
Step 1 Condition Statements (words)	Step 3 Condition Entries (Y/N)
Step 2 Action Statements (words)	Step 4 Action Entries (X)

Creating a Decision Table

Condition Heading	Decision Rule Number
Step 1 Condition Statements (words)	Step 3 Condition Entries (Y/N)
Step 2 Action Statements (words)	Step 4 Action Entries (X)

Example 1 - Identify Conditions and Actions

- Case Scenario

- If it is 7 am, you have to get up to work on time if it is on week days (Monday to Friday) only. If it is a weekend, you stay in bed even though it is already 7 am.

- Summary List of Conditions (2)

C1 = Is it 7am ?

C2 = Is it week day ?

- Summary List of Actions (2)

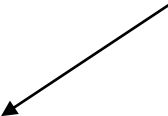
A1 = Get up to work

A2 = Stay in bed

Calculate the Number of Combinations

- Using Half Rules :
 - The number of combinations needed can be calculated by the following formula :

No. of conditions


$$2^C = 2^2 = 4 \text{ combinations}$$

2. Constructing a Decision Table

No. of conditions

$$2^C = 2^2 = 4 \text{ combinations}$$

The number of combinations = 4

Step 1

List all conditions

List all Actions

	1	2	3	4
Is it Saturday ?	Y	Y	N	N
Is it raining ?	Y	N	Y	N
Go visit friends	X	X		
Play tennis				X
Study			X	

Step 3

4/2 values = 2

2/2 values = 1

Action entries based on scenario

Step 2

Step 4

Member type : 3 values

some book –haven't return : 2 values

3*2=6 columns

6/3=2

2/2=1

Academic staff : tutor

Non Academic staff : librarian , accounting clerk

Student Student Astaff Astaff NAstaff NAstaff

Y

N

Y

N

Y

N

Completed Decision Table

The number of
combinations = 4

		1	2	3	4
Step 1	Is it 7 o'clock yet ?	Y	Y	N	N
	Is it a weekday ?	Y	N	Y	N
Step 2	Get up to work	X			
	Stay in bed		X	X	X

Step 3
4/2 values = 2
2/2 values = 1

Step 4
Action entries
based on scenario

Example 2 – Identifying Conditions & Actions

- Mutu Samy is a student studying for an engineering examination.
- If it is **Saturday night** he will not study, but **go visit friends**.
- (If) on other nights (ie not Saturday night)
 - If it **rains**, he will **study** at home.
 - When (if) it does not rain, he **plays tennis**.



3 Actions



2 Conditions

Steps Involved

- (a) Identify the number of conditions (2)
 - C1 = Is it Saturday night ? 2 values
 - C2 = is it raining ? 2 values
 - $2*2 = 4$ columns
- (b) Identify the number of actions (3)
 - A1 = Go visit friends
 - A2 = Study
 - A3 = Plays tennis
- (c) Calculate the number of combinations (4)
 - The number of conditions needed can then be calculated by the formula :


 $2^C = 2^2 = 4 \text{ combinations}$

Steps Involved (cont..)

Ensure the Conditions and **Actions** are listed in SEQUENCE

4 combinations (2^2)

Conditions	1	2	3	4
Is it Saturday night ?	Y	Y	N	N
Is it raining ?	Y	N	Y	N
Go visit friends				
Study				
Play tennis				

$4/2 = 2$
 $2/2 = 1$

Steps Involved

- (e) Completion of action entry
 - Next entering Xs against the appropriate actions completes the action entry. This also is illustrated in the following table :

Conditions	1	2	3	4
Is it Saturday night ?	Y	Y	N	N
Is it raining ?	Y	N	Y	N
Go visit friends	X	X		
Study			X	
Play tennis				X

$$4/2 = 2$$

$$2/2 = 1$$

Steps Involved

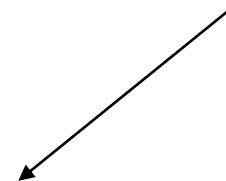
- (f) Removal of redundant/repeat conditions

Conditions	1	2	3	4
Is it Saturday night ?	Y	Y	N	N
Is it raining ?	Y	N	Y	N
Go visit friends	X	X		
Study			X	
Play tennis				X



Conditions	1-2	3	4
Is it Saturday night ?	Y	N	N
Is it raining ?	-	Y	N
Go visit friends	X		
Study		X	
Play tennis			X

Conditions	1	2	3
Is it Saturday night ?	Y	N	N
Is it raining ?	-	Y	N
Go visit friends	X		
Study		X	
Play tennis			X



Exercise 2 for Students

Sales Promotion Policy

Customers can be divided into two categories: **preferred customers** or not preferred customers.

Preferred customers who **order \$2,000 or more**, are entitled to a **6% discount**, and an **additional 5% discount** if they **use the company's charge card**.

Preferred customers who order less than \$2,000 will receive a **\$20 bonus coupon**. All other customers and all other circumstances, receive a **\$6 bonus coupon**.

1. Identify Conditions and Actions

Sales Promotion Policy

Customers can be divided into two categories:
preferred customers or not preferred customers.

Preferred customers who **order \$2,000 or more**, are entitled to a **6% discount**, and an **additional 5% discount** if they **use the company's charge card**.

Preferred customers who order less than \$2,000 will receive a **\$20 bonus coupon**. All other customers and all other circumstances, receive a **\$6 bonus coupon**.

C = Conditions

A = Actions

Steps Involved

- (a) Identify the number of conditions
 - Preferred customers ? 2
 - Ordered $\geq$ \$2,000 ? 2
 - Used charge card ? 2
- (b) Identify the number of actions
 - 6% discount
 - Additional 5% discount
 - \$20 bonus coupon
 - \$6 bonus coupon
- (c) Calculate the number of combinations
 - The number of conditions needed can then be calculated by the formula :
 $2 \times 2 \times 2 = 8$

No. of conditions

3

$$2^3 = 8 \text{ combinations}$$

Exercise 2

[illegible]

1. Identify Conditions and Actions

Sales Promotion Policy

Customers can be divided into two categories:
preferred customers or not preferred customers.

Preferred customers who **order \$2,000 or more**, are entitled to a **6% discount**, and an **additional 5% discount** if they **use the company's charge card**.

Preferred customers who order less than \$2,000 will receive a **\$20 bonus coupon**. All other customers and all other circumstances, receive a **\$6 bonus coupon**.

C = Conditions

A = Actions

Suggested Answer 2

Conditions	1	2	3	4	5	6	7	8
Preferred customer ?	Y	Y	Y	Y	N	N	N	N
Ordered $\geq$ \$2,000 ?	Y	Y	N	N	Y	Y	N	N
Used charge card ?	Y	N	Y	N	Y	N	Y	N
6% discount	X	X						
Additional 5% discount	X							
\$20 bonus coupon			X	X				
\$6 bonus coupon					X	X	X	X

Suggested Answer 2

Conditions	1	2	3	4	5	6	7	8
Preferred customer ?	Y	Y	Y	Y	N	N	N	N
Ordered $\geq$ \$2,000 ?	Y	Y	N	N	Y	Y	N	N
Used charge card ?	Y	N	Y	N	Y	N	Y	N
6% discount	X	X						
Additional 5% discount	X							
\$20 bonus coupon			X	X				
\$6 bonus coupon					X	X	X	X

Conditions	1	2	3	4	5	6	7	8
Preferred customer ?	Y	Y	Y	Y	N	N	N	N
Ordered >= \$2,000 ?	Y	Y	N	N	Y	Y	N	N
Used charge card ?	Y	N	Y	N	Y	N	Y	N
6% discount	X	X						
Additional 5%discount	X							
\$20 bonus coupon			X	X				
\$6 bonus coupon					X	X	X	X

conditions	1	2	3	4
Preferred customer ?	Y	Y	Y	N
Ordered >= RM1,000 ?	Y	Y	N	-
Used our charge card ?	Y	N	-	-
6% discount	X	X		
Additional 5%discount	X			
\$20 bonus coupon			X	
\$6 bonus coupon				X

Exercise 3 for Students (try yourself)

- This case is concerned with provision of discounts to customers.
- Mr. John will not allow any discounts if both the following conditions are not fulfilled. However, **8% discount** will be given if both the conditions are fulfilled.
 - the **balance** on the account is less than \$300
 - all items **outstanding** are less than 4 months old
- If one of these conditions is violated, the **discount allowed is 4%**. Customers who have been **trading for more than 3 years** are allowed an **additional 2%** provided that none of the above conditions are violated.

Suggested Answers 3

Conditions	1	2	3	4	5	6	7	8
Balance < \$300 ?	Y	Y	Y	Y	N	N	N	N
Outstand < 4 months ?	Y	Y	N	N	Y	Y	N	N
Trading > 3 years ?	Y	N	Y	N	Y	N	Y	N
No discount								
8% discount								
4% discount								
Additional 2% discount								

Exercise 3 for Students (try yourself)

- This case is concerned with provision of discounts to customers.
- Mr. John will not allow any discounts if both the following conditions are not fulfilled. However, **8% discount** will be given if both the conditions are fulfilled.
 - the **balance** on the account is less than \$300
 - all items **outstanding** are less than 4 months old
- If one of these conditions is violated, the **discount allowed is 4%**. Customers who have been **trading for more than 3 years** are allowed an **additional 2%** provided that none of the above conditions are violated.

Suggested Answers 3

Conditions	1	2	3	4	5	6	7	8
Balance < \$300 ?	Y	Y	Y	Y	N	N	N	N
Outstand < 4 months ?	Y	Y	N	N	Y	Y	N	N
Trading > 3 years ?	Y	N	Y	N	Y	N	Y	N
No discount							X	X
8% discount	X	X						
4% discount			X	X	X	X		
Additional 2% discount	X							

Suggested Answers 3 (cont..)

(f) Removal of redundant conditions

- Redundant conditions = different conditions which produce similar actions.
- Example - Rule 3 & 4, 5 & 6, and 7 & 8 are redundant because each set of rules produce the same results.
- Rules can be combined together systematically, two columns at a time.
- The two columns can be written down, as one with a hyphen (-) replacing the entries in the conditions.

Suggested Answers 3

Conditions	1	2	3	4	5	6	7	8
Balance < \$300 ?	Y	Y	Y	Y	N	N	N	N
Outstand < 4 months ?	Y	Y	N	N	Y	Y	N	N
Trading > 3 years ?	Y	N	Y	N	Y	N	Y	N
No discount							X	X
8% discount	X	X						
4% discount			X	X	X	X		
Additional 2% discount	X							

Conditions	1	2	3,4	5,6	7,8
Balance < \$300 ?	Y	Y	Y	N	N
Outstand < 4 months ?	Y	Y	N	Y	N
Trading > 3 years ?	Y	N	-	-	-
No discount					X
8% discount	X	X			
4% discount			X	X	
Additional 2% discount	X				

Suggested Answers 3

- (g) Table with redundant conditions removed

Conditions	1	2	3,4	5,6	7,8
Balance < \$300 ?	Y	Y	Y	N	N
Outstand < 4 months ?	Y	Y	N	Y	N
Trading > 3 years ?	Y	N	-	-	-
No discount					X
8% discount	X	X			
4% discount			X	X	
Additional 2% discount	X				

Conditions	1	2	3	4	5
Balance < \$300 ?	Y	Y	Y	N	N
Outstand < 4 months ?	Y	Y	N	Y	N
Trading > 3 years ?	Y	N	-	-	-
No discount					X
8% discount	X	X			
4% discount			X	X	
Additional 2% discount	X				

Exercise 4 (try yourself)

- There are three conditions which might be encountered by a sales order processing clerk taking a telephone order.
 - The caller may have an **existing overdue balance** their account.
 - The caller's account balance may already be in **excess of their credit limit**.
 - The caller may not be an **not have an overdue balance** and have **existing customer** at all.
- **Orders from existing customers who do not exceeded their credit limit should be processed.**
- The organization decides that customers requiring an account must **provide credit references** for checking before the account can be opened. If these customers attempt to place an order before an account is set up, the **order is placed on hold**.
- The policy for orders from existing customers with a balance in excess of their credit limit is that these orders are placed on hold and **referred to the section head**.
- The policy for existing customers who have overdue balances, but are within their credit limit, is to process the order but to also **generate a reminder letter** for overdue balances.

Exercise 4

Conditions	1	2	3	4	5	6	7	8
Account overdue ?	Y	Y	Y	Y	N	N	N	N
Credit limit exceeded ?	Y	Y	N	N	Y	Y	N	N
An existing customer ?	Y	N	Y	N	Y	N	Y	N
Process order								
Obtain reference								
Place order on hold								
Refer to head								
Send reminder								

Exercise 4 (try yourself)

- There are three conditions which might be encountered by a sales order processing clerk taking a telephone order.
 - The caller may have an **existing overdue balance** their account.
 - The caller's account balance may already be in **excess of their credit limit**.
 - The caller may not be an **not have an overdue balance** and have **existing customer** at all.
- **Orders from existing customers who do not exceeded their credit limit should be processed.**
- The organization decides that customers requiring an account must **provide credit references** for checking before the account can be opened. If these customers attempt to place an order before an account is set up, the **order is placed on hold**.
- The policy for orders from existing customers with a balance in excess of their credit limit is that these orders are placed on hold and **referred to the section head**.
- The policy for existing customers who have overdue balances, but are within their credit limit, is to process the order but to also **generate a reminder letter** for overdue balances.

Suggested Answers 4

Conditions	1	2	3	4	5	6	7	8
Account overdue ?	Y	Y	Y	Y	N	N	N	N
Credit limit exceeded ?	Y	Y	N	N	Y	Y	N	N
An existing customer ?	Y	N	Y	N	Y	N	Y	N
Process order			X				X	
Obtain reference		X		X		X		X
Place order on hold	X	X		X	X	X		X
Refer to head	X				X			
Send reminder			X					

Suggested Answers 4

Conditions	1,5	2,4,6,8	3	7
Account overdue ?	-	-	Y	N
Credit limit exceeded ?	Y	-	N	N
An existing customer ?	Y	N	Y	Y
Process order			X	X
Obtain reference		X		
Place order on hold	X	X		
Refer to head	X			
Send reminder			X	

Conditions	1	2	3	4
Account overdue ?	-	-	Y	N
Credit limit exceeded ?	Y	-	N	N
An existing customer ?	Y	N	Y	Y
Process order			X	X
Obtain reference		X		
Place order on hold	X	X		
Refer to head	X			
Send reminder			X	

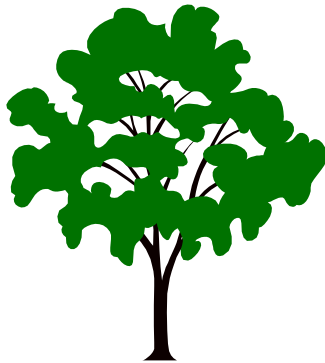
Advantages of Using Decision Table

- **Complex Problems.** When the problem situation is large, decision tables are easy and useful.
- **System Testing.** Decision tables are also useful for designing test scripts for system testing, ensuring all possible combinations of testing conditions and actions are taken into account.

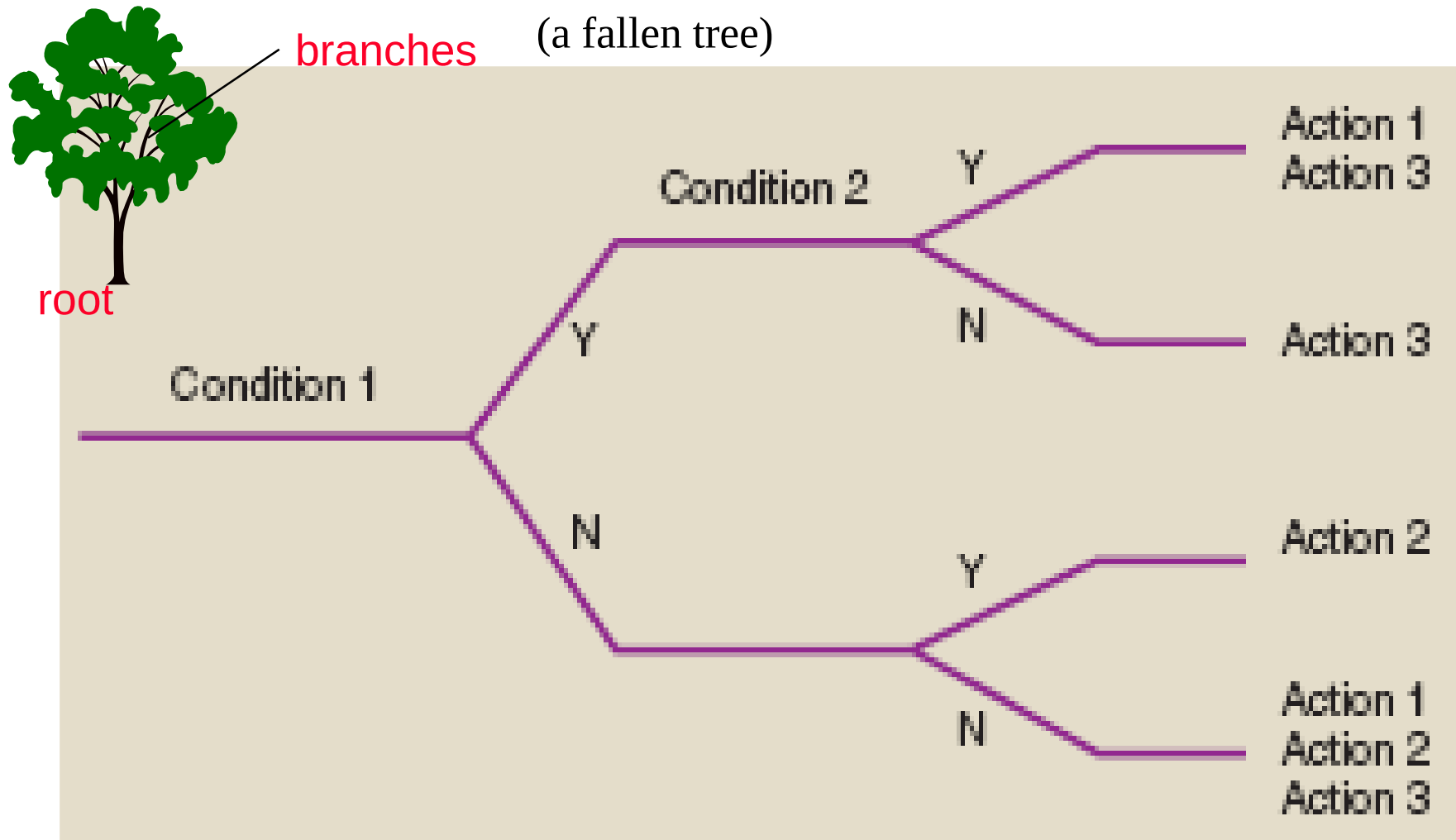
Decision Tree

Decision Trees

- Decision trees offer an **alternative** to structured English or decision tables as a means of **defining complex relationships**.
- A decision tree is a **design tool** which provides **graphical representation** of the choices or decisions made in a particular processes (in the form of a 'fallen' tree).

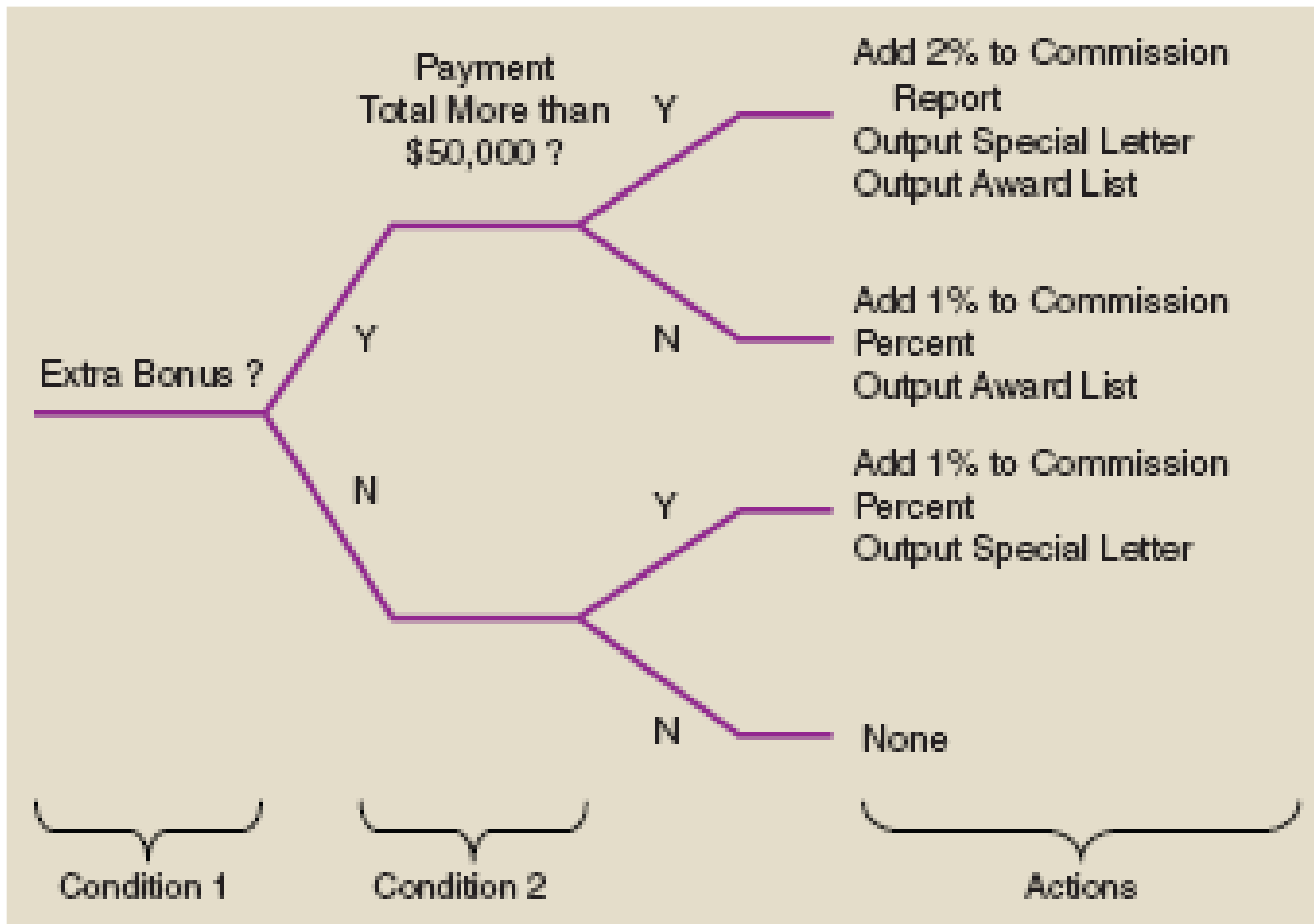


General Format of Decision Tree



Note : the conditions and actions

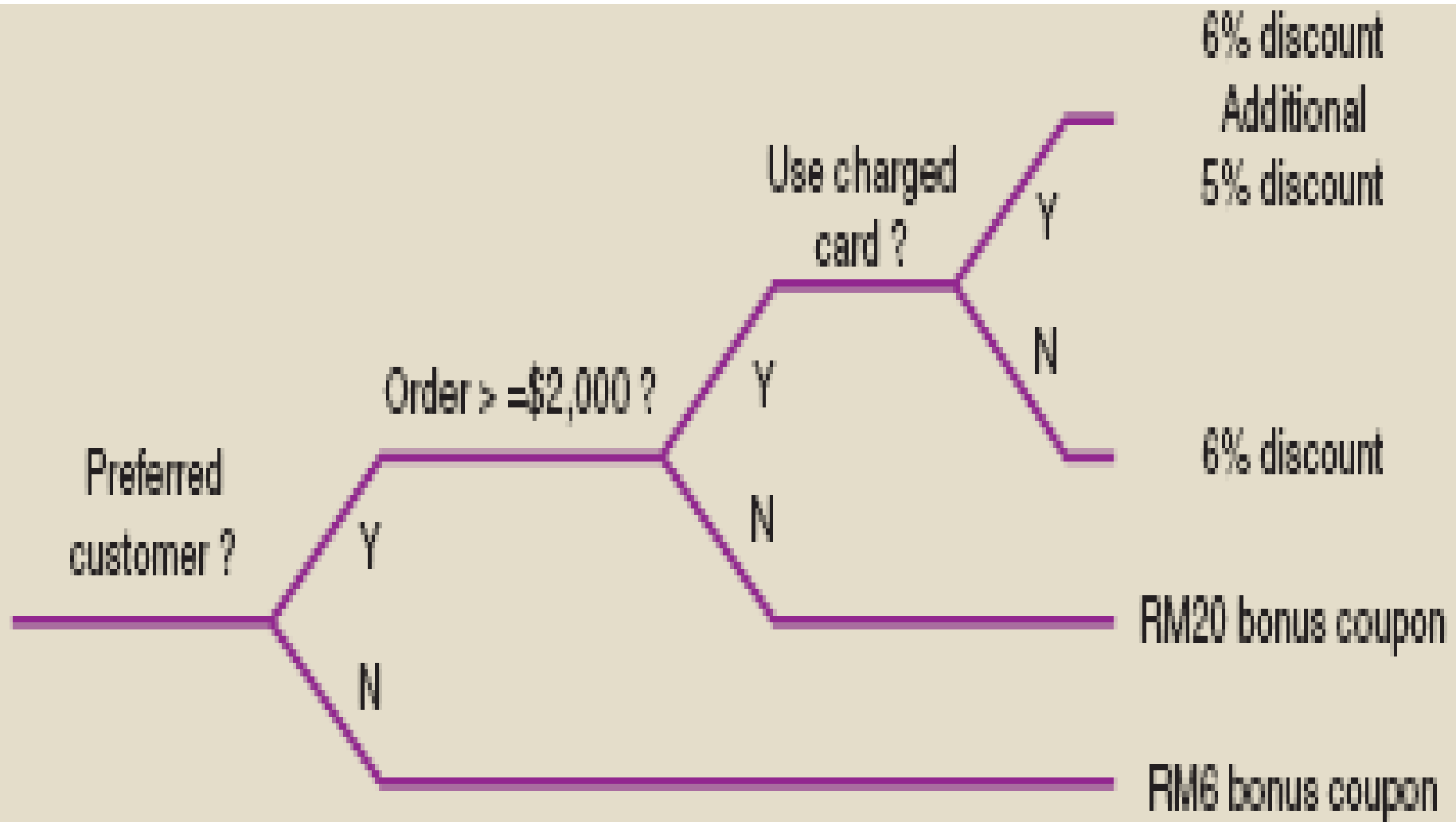
Decision Tree - Example



Decision Tree – Exercise 1

- Sales promotion policy.
- Customers can be divided into two categories: **preferred** customers or not preferred customers. For preferred customers who **order RM2,000** or more, they are entitled to a **6% discount**, and an additional **5% discount** if they use the company's **charge card**. Preferred customers who order less than \$2,000 will receive a **RM 20 bonus** coupon. All other customers and all other circumstances, receive a **RM6 bonus** coupon.

Solution 1



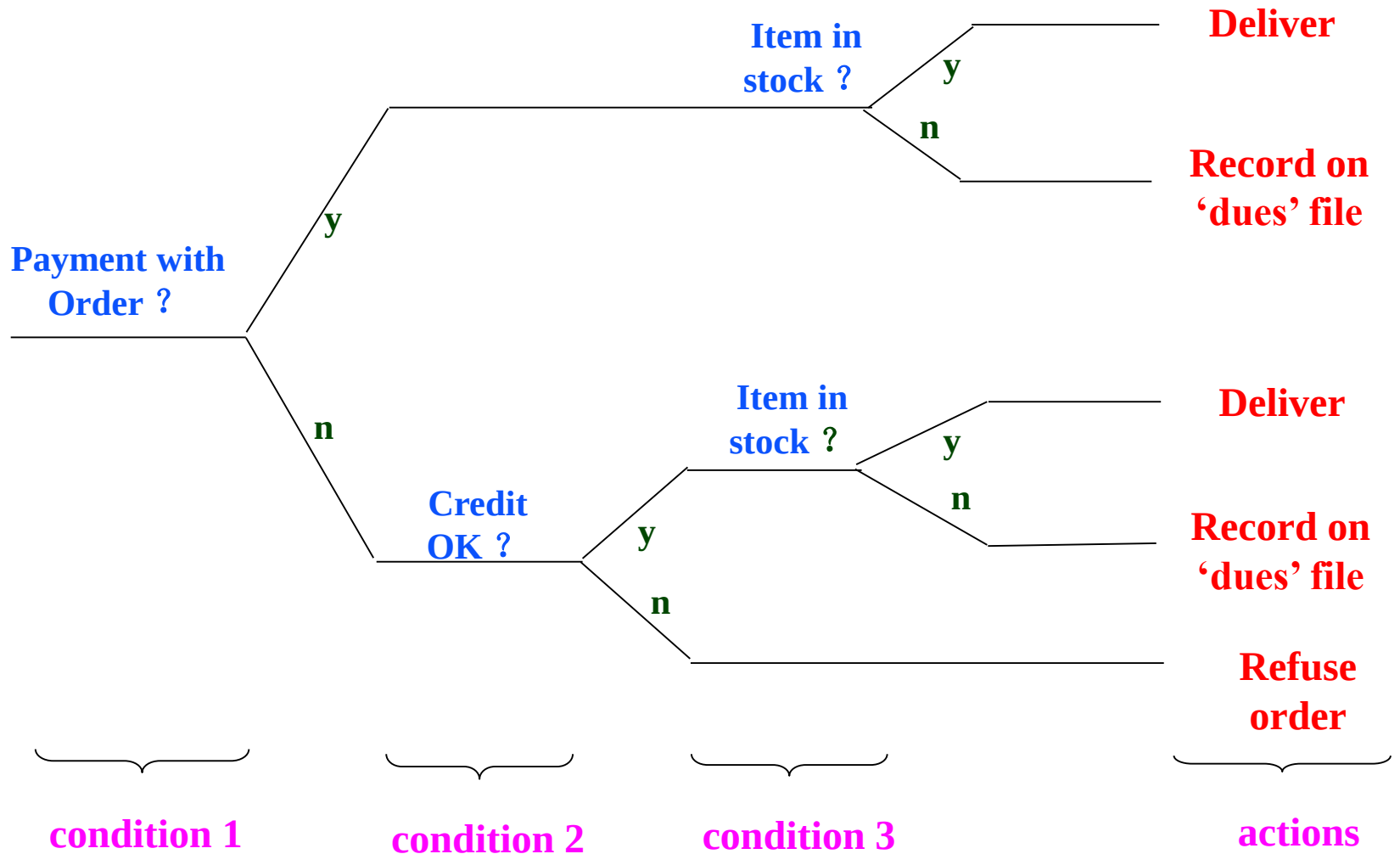
Exercise 2 – Decision tree (try yourself)

Order Processing System

Whenever an order is received the system should first check to determine if payment has been received with the order. There are two possibilities – yes and no – which lead on to differing responses.

- If payment is included with the order, the system would then have to check on the availability of the items ordered. Here there are two further possibilities – the items can be in stock or not.
 - ❖ If the item are in stock they would then be shipped or delivered to the customer.
 - ❖ If the item is not in stock a record of the customer's order ~~orders~~ to be made so that the item will be despatched when stock becomes available.
- If the customer has not included payment with order a similar/same set of activities would be undertaken but they would be preceded by credit evaluations first.

Solution 2

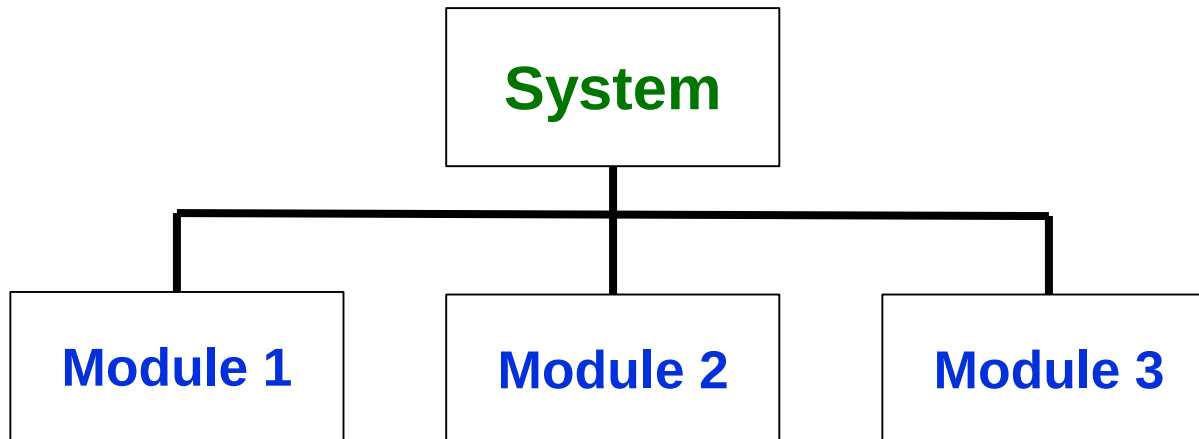


Structured Program Design

(To make a program more structured)

Structured Program Design

- **Introduction**



Modularity (High)

- **System**

Coupling (Low)

- Modules

Cohesion (High)

- Codes

[illegible]

```
DO calculate_sales-tax
  IF item is exempted from tax THEN
    sales_tax = 0
  ELSE
    IF item_price < RM100 THEN
      sales_tax = item_price * 0.3
    ELSE
      IF item_price <= RM200 THEN
        sales_tax = item_price * 0.4
      ELSE
        sales_tax = item_price * 0.5
      ENDIF
    ENDIF
  ENDIF
ENDIF
ENDDO
```

```

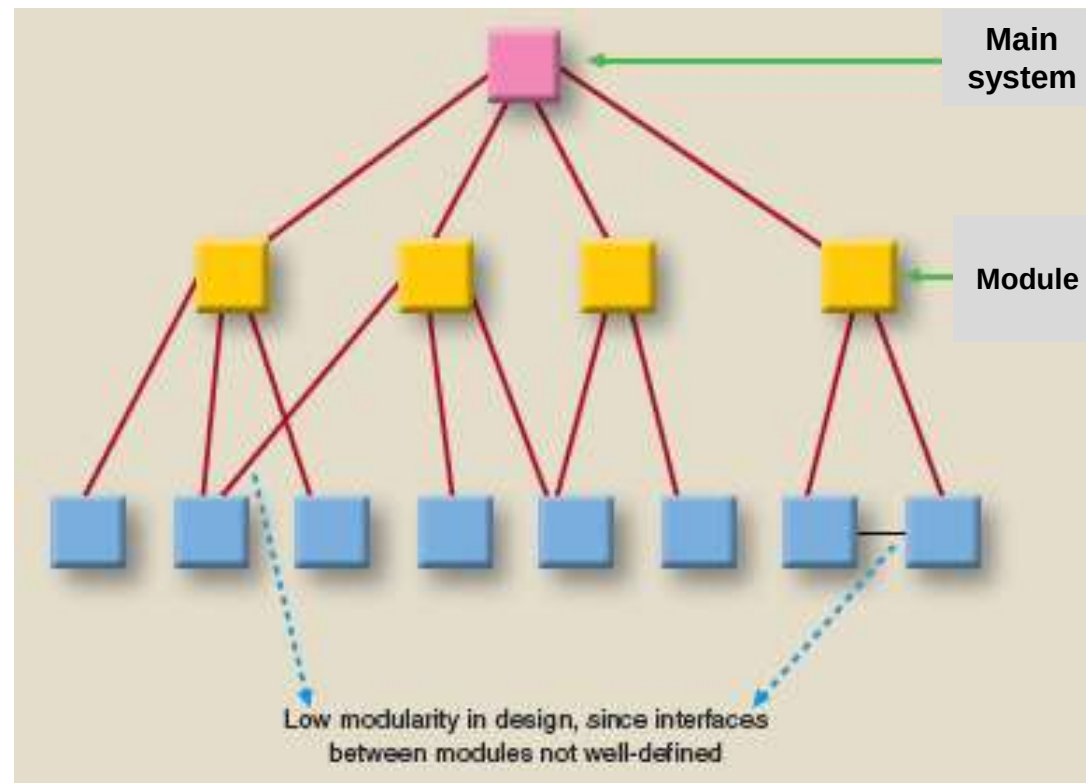
IF ALL ZEROES
  SCALE BY THE ZERO
  READ NUMBER OF TRANSDUCER INPUTS
  IF NOT EOF
    FOR COUNTER = 1 TO NUMBER OF TRANSDUCER INPUTS
      READ CONFIRMED COUNTER INPUT
      IF NOT EOF
        INCREMENT SURVEYOR NUMBER AGAIN
      ENDIF
    ENDFOR
  ENDIF
  READ PUNCH 1 COUNTER NUMBER
  IF NOT EOF
    FOR COUNTER = 1 TO NUMBER OF TRANSDUCER INPUTS
      READ PUNCH 1 COUNTER NUMBER
    ENDFOR
  ENDIF
  PRINT COUNTER

```

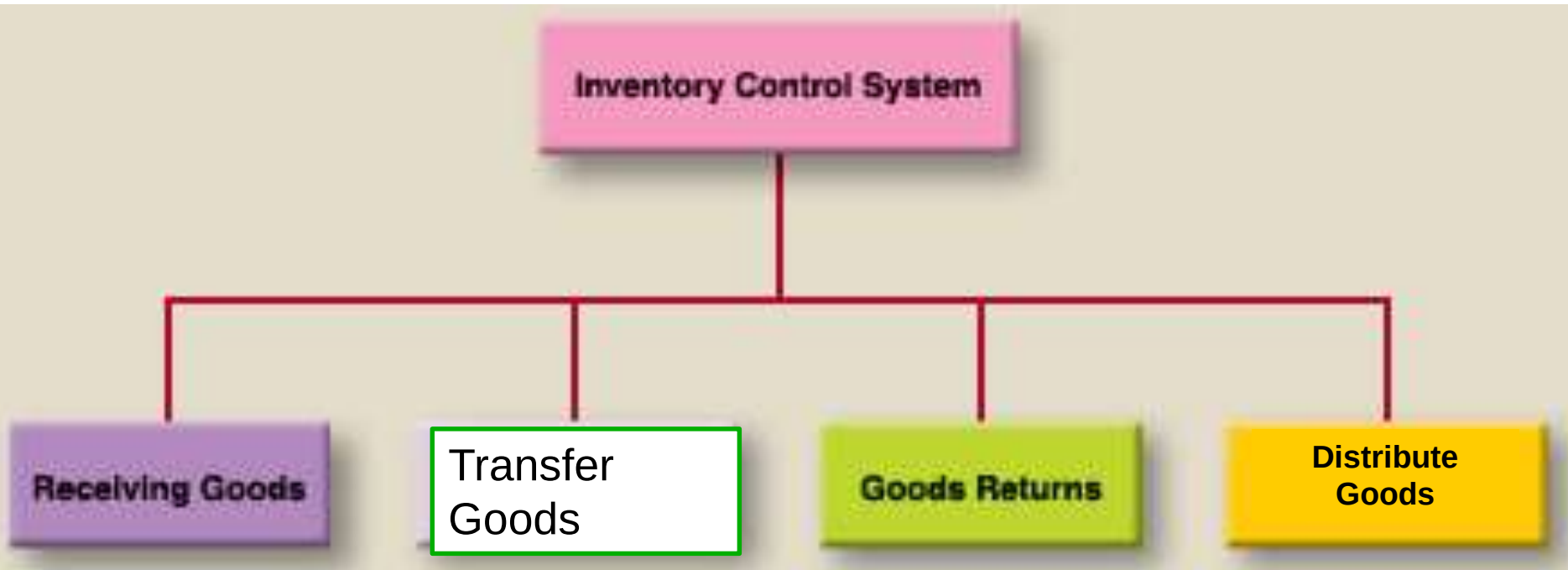
1. Modularity

- **Definition** - the system is partitioned into simpler **sub-systems** or modules. Each module is designed to perform a particular activity/task.
- **Benefits** – easy to understand, code and maintain the system.

Modularity (**High**)
- **System**



Example – Inventory Control System



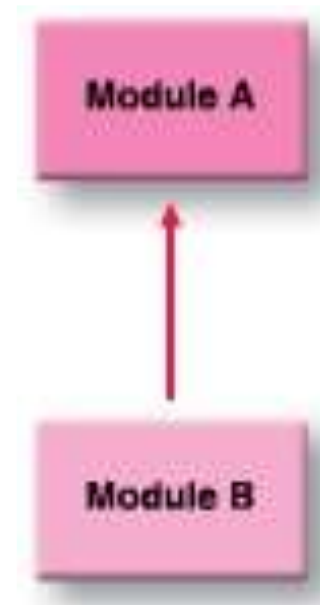
Modularity (High)
- **System**

Receive Goods, Transfer Goods,
Distribute Goods/Issues Goods,
Return Goods

2. Coupling

- **Definition** – degree of *dependence* between modules. High coupling means the modules are tightly connected (passing of data).
- **Implications :**
 - A bug in one module can infect other module.
 - If one module is modified, the other may also need to be modified.
- **Example - Tight Coupling**

Coupling (**Low**)
- **Modules**



Loose
dependenc
y between
Module A
and
Module B



3. Cohesion

- Definition – measures the **functional relationship** of the *instructions* exists within a module. Cohesive if all the instructions in the module jointly accomplish a **single** task.
- Implications
 - More efficient.
 - Well structured, easy to code and debug.

High Cohesiveness in programming to calculate sales-tax module

In this module, every single line of code is written to perform the single ONE task ie to calculate **Sales_Tax**.

Cohesion (**High**)

- **Codes**

```
DO calculate_sales-tax

IF item is exempted from tax THEN
    sales_tax = 0
ELSE
    IF item_price < RM100 THEN
        sales_tax = item_price * 0.3
    ELSE
        IF item_price < RM200 THEN
            sales_tax = item_price * 0.4
        ELSE
            sales_tax = item_price * 0.5
        ENDIF
    ENDIF
ENDIF
ENDIF
ENDDO
```


Low Cohesiveness used in performing record updating

In this module, the 3 instructions perform 3 **separate** functions.

```
MODULE UPDATE
  BEGIN
    READ AT THE BASE
    READ customer transaction record
    IF not EOP
      Increment customer_transaction_count
    ENDIF
    READ customer record record
    IF not EOP
      INCREMENT customer_record_count
    ENDIF
    IF &N greater 1 record count
      Increment greater 1 record_count
    ENDIF
  ENDMODULE
```

Exercise for Students

- Explain the **THREE (3)** characteristics of system design which will enable the system to be more structured, easier to code, understand, debug and maintain.

Solution

- **Characteristics of a Good Design :**
 - 1. High modularity** – the systems is broken down into smaller modules
 - 2. Low coupling** – the modules should not be too connected or related with each other.
 - 3. High cohesion** – the codes in a module should be programmed to perform a single task.

System Design Specification

- A set of document used to **record details of design.**
- Prepared by **System Analyst**

